Code review: Week 1

HAMM – Counting point mutation

Doyoung LEE

Hamming distance (1)

- 길이가 같은 두 서열을 같은 위치끼리 1:1로 비교했을 때 서로 다른 문자의 개수
  - 두 DNA 서열 사이의 point mutation 수를 세는 척도로 사용
  - 두 서열이 같은 길이이고 정렬(alignment)이 이미 되어 있다고 가정
  - SNP 등의 point mutation만 count



Figure 2. The Hamming distance between these two strings is 7. Mismatched symbols are colored red.

Hamming distance (2)

- 삽입/결실(indel)은 다루지 못함
  - 한 염기가 삽입 또는 결실되면 그 뒤가 전부 밀림
  - 거의 모든 위치가 mismatch로 계산
  - indel까지 고려한 거리는 Levenshtein distance로 계산 가능

- 응용
  - sequencing read 간 mismatch 계산
  - 다형성(polymorphism) 비교
  - 약물 표적 유전자의 변이 비교

Computational biology 핵심 개념: SNP와 MSA

단일 염기 다형성 (SNP)

Single Nucleotide Polymorphism

- **정의** 게놈의 한 위치에서 염기 하나가 집단 내에서 달라지는 변이. 관례상 소수 대립유전자 빈도(MAF) ≥ 1%
- **기능** 코딩 영역은 synonymous / nonsynonymous로 구분. 비코딩 영역도 발현·splicing에 영향 가능
- **규모** 1000 Genomes Phase 3: 2,504명, 26개 집단에서 SNP 약 8,470만 개
- **활용** array 설계, genotype imputation, 질병 연관 연구(GWAS)의 기반 자료

예시 (가상 서열): 이 위치의 대립유전자는 G 또는 A

```
Ref   …ACGGTC…
Ind   …ACGATC…
          ^
```

다중 서열 정렬 (MSA)

Multiple Sequence Alignment

- **정의** 3개 이상의 서열(단백질·DNA·RNA)을 정렬해 진화적으로 대응되는 위치를 같은 열에 맞춤
- **난제** sum-of-pairs(SP) 점수 최적화는 NP-complete라서 대부분 휴리스틱을 사용
- **방법** Progressive(guide tree 순서로 정렬) 후 iterative refinement. 도구: Clustal Omega, MAFFT, MUSCLE

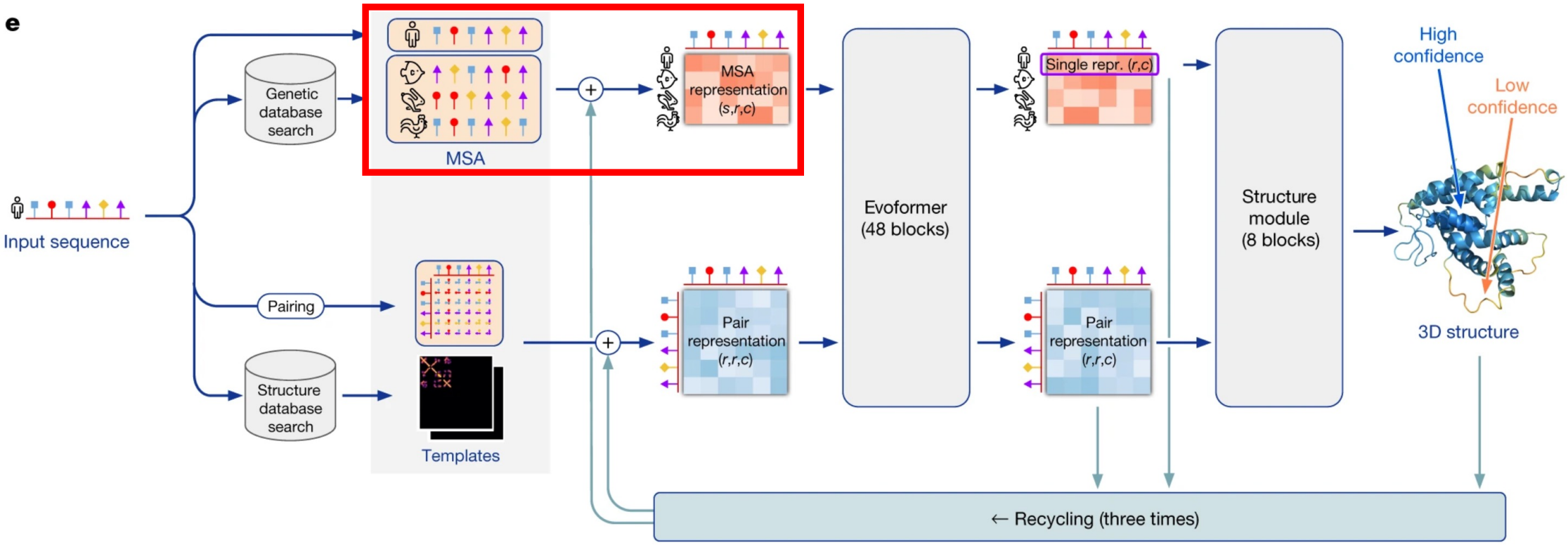
예시 (가상 서열)

```
S1   ATGCTAAG
S2   ATGC-AAG
S3   AT-CTAAG
S4   ACGCTATG
     *  * * *
```

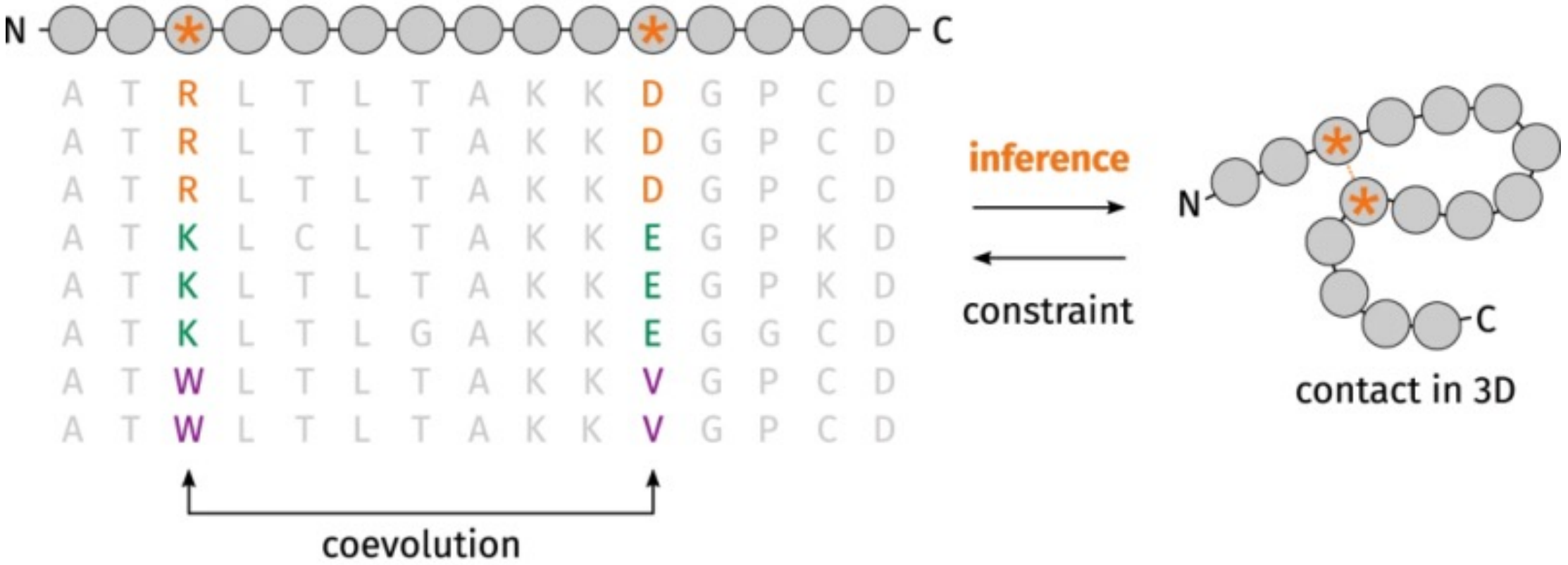
출처: 1000 Genomes Consortium, Nature 2015 · Chatzou et al., Brief Bioinform 2016 · Edgar, BMC Bioinformatics 2004

Multiple Sequence Alignment (MSA)

- AlphaFold2 (https://www.nature.com/articles/s41586-021-03819-2)



Multiple Sequence Alignment (MSA)



Code review

- 입력으로 받은 두 DNA 문자열 s, t를 준비
- 두 문자열의 **같은 인덱스에 있는 문자끼리** 비교
- **다른 경우의 수**를 카운트
- 그 숫자를 출력

Doyoung_HAMM.ipynb > hamm = 0

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ... | base (Python 3.12.12)

Generate | Code | Markdown

```
a = 'GAGCCTACTAACGGGAT'
b = 'CATCGTAATGACGGCCT'
print(list(zip(a, b)))
```

[12] ✓ 0.0s Python

```
[('G', 'C'), ('A', 'A'), ('G', 'T'), ('C', 'C'), ('C', 'G'), ('T', 'T'), ('A', 'A'), ('C', 'A'), ('T', 'T'), ('A', 'G'), ('A', 'A
```

```
hamm = 0

for (x, y) in zip(a, b):
    if x == y:
        continue
    else:
        hamm += 1

print (hamm)
```

[13] ✓ 0.0s Python

```
7
```

Any Question?